Tipos, tiempo y pulsadores

El **tamaño real** de cada tipo importa, el tiempo se mide **sin bloquear** (`millis()` + resta) y un pulsador se lee con **pull-up + antirrebote + flanco**.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 El patrón millis()

mirar el reloj de pared, no esperar

1 — Leer el reloj
Cada vuelta del `loop()`.
`ahora = millis()`

2 — ¿ahora − tAnterior ≥ PERIODO?

Sí → **Actuar y anotar**
Hacer algo y seguir al paso 3.
`tAnterior += PERIODO`

No → **3 — Seguir con lo demás**
Leer botones, etc.: hay tiempo.

↻ vuelve al paso 1: el `loop()` nunca se bloquea

DELAY() BLOQUEA

Es el cocinero que mira el horno: mientras espera, el programa no hace nada más y una pulsación se pierde. `millis()` solo **mira** el reloj; el contador de hardware cuenta siempre (timers: semana 3).

REGLA DE ORO

`ahora - tAnterior >= PERIODO`, siempre con `uint32_t`. Restar funciona aunque el contador dé la vuelta: reloj de 8 bits, `tAnterior = 250`, `ahora = 4` → 4 − 250 = 10 (mód 256), justo el tiempo que pasó. ✘ `millis() > tAnterior + PERIODO` falla: la suma da la vuelta.

TAMAÑOS: INT NO MIDE LO MISMO EN TODAS PARTES (BYTES)

TIPO	AVR (UNO)	ESP32	OJO
`char`	1 · con signo	1 · sin signo	Usar `int8_t` / `uint8_t`
`short`	2	2	—
`int`	**2**	**4**	¡Cambia!
`long`	4	4	—
`float`	4	4 (FPU)	Rápido en el ESP32
`double`	4 (= float)	8 (software)	Lento en el ESP32
puntero	2	4	¡Cambia de tamaño!

2 Antirrebote + flanco, paso a paso

un toque = un conteo

1 — Leer el pin

`lectura = digitalRead(PIN_BTN)`

2 — ¿Cambió respecto a la lectura anterior?

Sí → **Reiniciar el cronómetro**
Y seguir al paso 3.
`tUltimoCambio = millis()`

No

3 — ¿Quieta ≥ 30 ms y distinta del estado estable?

No → **Volver a leer**
Todavía puede ser rebote: paso 1.

Sí

4 — Aceptar el nuevo estado estable

Es un cambio real.
`estadoEstable = lectura`

5 — ¿Es LOW? Flanco de bajada

Sí → **Contar la pulsación**
Una sola vez por toque.
`pulsaciones++`

No, se soltó

vuelve al paso 1

CÓDIGO: ANTIRREBOTE + FLANCO

```
constexpr uint8_t PIN_BTN = 21;
constexpr uint32_t T_REBOTE = 30;   // ms
bool lecturaAnterior = HIGH, estadoEstable = HIGH;
uint32_t tUltimoCambio = 0; uint16_t pulsaciones = 0;

void setup() { Serial.begin(115200); pinMode(PIN_BTN, INPUT_PULLUP); }

void loop() {
  bool lectura = digitalRead(PIN_BTN);
  if (lectura != lecturaAnterior) {          // se movió: reiniciar
    tUltimoCambio = millis(); lecturaAnterior = lectura;
  }
  if (millis() - tUltimoCambio >= T_REBOTE && lectura != estadoEstable) {
    estadoEstable = lectura;                 // cambio real
    if (estadoEstable == LOW) {              // flanco de bajada
      pulsaciones++; Serial.println(pulsaciones);
    }
  }
}
```

TRES IDEAS DEL PULSADOR

- **Pull-up:** suelto = HIGH (la pull-up interna lleva el pin a 3,3 V); presionado = LOW (el botón une el pin con GND). GPIO 34–39: sin pull-up interna.
- **Rebote:** un botón mecánico produce varios cambios falsos en pocos ms; acepta el cambio si la señal queda quieta **20–50 ms**. ✘ `delay(200)` vuelve a bloquear.
- **Nivel vs. flanco:** un toque dura ~200 ms, y el nivel LOW dura miles de vueltas. ✘ `if (digitalRead(BTN) == LOW) contador++;` suma miles. ✔ Actuar **solo en el flanco** HIGH → LOW.

3 Entradas → lógica → salidas

la estructura de un juego de luces

1. **Entradas** — botón con antirrebote y flanco — `botonPresionado()`
2. **Lógica** — estado + `millis()`: ¿toca avanzar? — `ahora - tPaso >= PERIODOS[v]`
3. **Salidas** — LEDs según el estado — `digitalWrite(LEDS[i], …)`

↻ vuelve a las entradas en cada vuelta del `loop()`

DESBORDAMIENTO: EL ODÓMETRO

`uint8_t c = 253;` y `c++` cinco veces: 253 254 255 **0** 1. **Sin signo** da la vuelta (definido); **con signo** es comportamiento indefinido. `int t = millis();` falla a los **~32 s** en el Uno y a los **~24,8 días** en el ESP32 → `uint32_t`.

EL 1 TAMBIÉN TIENE TIPO

`1 << 31` es un `int` con signo y desborda; `1UL << 31` es de 32 bits sin signo. Tras `~a`, los tipos chicos se promueven a `int`: usar el cast `(uint8_t)~a` para volver a 8 bits.

ACTIVIDAD «ENCUENTRA EL BUG» (RESUELTA)

CÓDIGO	BUG	CORRECCIÓN
`for (uint8_t i = 0; i < 256; i++)`	`uint8_t` nunca llega a 256 (255 → 0): **bucle infinito**	`uint16_t i`
`int t0 = millis();`	`millis()` devuelve `uint32_t`: el `int` desborda	`uint32_t t0`
`uint32_t m = 1 << 31;`	El `1` es `int` con signo: desborda	`1UL << 31`
`void loop() { int veces = 0; veces++; }`	Local: vuelve a 0 en cada vuelta	`static int veces = 0;`